

Instruction	Execution	Description
add R11, R10	R10 ← R10 + R11	Add
addc R11, R10	R10 ← R10 + R11 + C	Add with carry bit
sub R11, R10	R10 ← R10 - R11	Subtract
subc R11, R10	R10 ← R10 - R11 - ([NOT]C)	Subtract with carry bit
dadd R11, R10	R10 ← R10 + R11 + C (Decimally)	BCD addition
*inc R10	R10 ← R10 + 1	Increment by 1
*incd R10	R10 ← R10 + 2	Increment by 2
*dec R10	R10 ← R10 - 1	Decrement by 1
*decd R10	R10 ← R10 - 2	Decrement by 2
rra R10	MSB → MSB → ... LSB → C	Shift right (Arithmetically)
*rla R10	ADD R10, R10	Shift left (Arithmetically)

Instruction	Execution	Description
and R11, R10	R10 ← R10 (AND) R11	Bitwise AND
bit R11, R10	R10 ← R10 (OR) R11	Bitwise OR
xor R11, R10	R10 ← R10 (XOR) R11	Bitwise XOR
*inv R10	R10 ← (NOT) R10	Invert, Bitwise NOT
bic R11, R10	R10 ← R10 (AND) ([NOT] R11)	Clear bits
rrc R10	C → MSB → ... LSB → C	Shift right (Logical)
*rlc R10	ADDC R10, R10	Shift left (Logical through Carry)

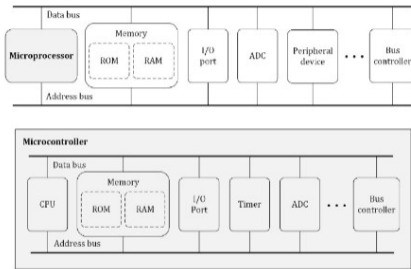
Instruction	Execution	Description
cnp R11, R10	Status flags on R10 - R11	Compare
tst R10	Status flags on R10 - 0	Test
bit R11, R10	Status flags on R10 (AND) R11	Test bits
jmp label	PC ← address in label	Unconditional branch
*br label	PC ← address in label	Unconditional branch
call label	SP ← SP-2; @SP ← PC; PC ← address in label	Call subroutine
*ret	PC ← @SP (16bit); SP ← SP+2	Return from subroutine
reti	PC ← @SP (20bit); SP ← SP+2	Return from interrupt

Instruction	Condition	Description
jeq label	equal	branch to label if Z bit is Set
jne label	not equal	branch to label if Z bit is Reset
jz label	zero	branch to label if Z bit is Set
jnz label	not zero	branch to label if Z bit is Reset
jc label	carry	branch to label if C bit is Set
jnc label	no carry	branch to label if C bit is Reset
jge label	greater or equal	branch to label if N and V bits are the same
jle label	less	branch to label if N and V bits are not the same

Instruction	Execution	Description
mov R11, R10	R10 ← R11	Register operation
mov #imm, R10	R10 ← immediate value (imm)	Immediate mode operation
mov @R11, R10	R10 ← @R11	Load (Indexed)
mov @R11, R10	R10 ← @R11	Load (Indirect register)
mov label, R10	R10 ← address in label	Load (Symbolic)
mov @label, R10	R10 ← address in label	Load (Absolute)
mov R11, @R10	@R10 ← R11	Store (Indexed)
mov R10, label	@(address in label) ← R10	Store (Symbolic)
mov R10, @label	@(address in label) ← R10	Store (Absolute)

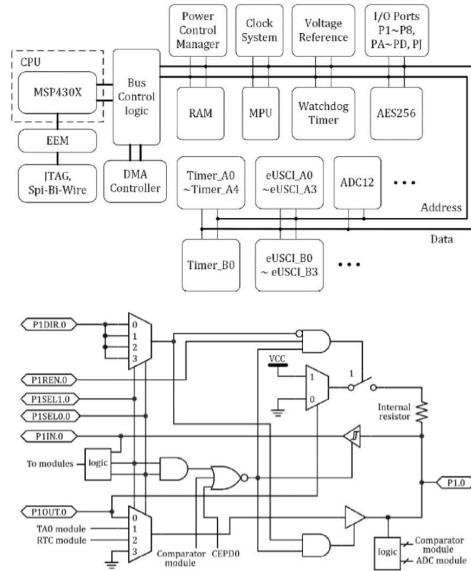
Instruction	Execution	Description
mov #label, R10	R10 ← address in label (16 bit)	Address of label to R10 (16 bits)
mov #label, R10	R10 ← address in label (20bit)	Address of label to R10 (20 bits)
push R10	SP ← SP-2; @SP ← R10	Push operation to the stack
*pop R10	R10 ← @SP; SP ← SP+2	Pop operation from stack
*nop	mov R3, R3	No operation

Microcontroller vs Microprocessor



	MSP430™ MCUs	Arm®-based MCUs	C2000™ MCUs	Wireless connectivity MCUs
Features	TI proprietary 16-bit MCUs Ultra-low power	Arm-based 32-bit MCUs	TI proprietary 32-bit MCUs or Arm-based 32-bit MCUs	Wireless Arm-based or TI proprietary MCUs
Models	MSP430x2xxx MSP430x4xxx MSP430x5xxx MSP430x6xxx	TM4Cxxx MSP432Pxxx MSP432Exxx	TMS320xxx	CC1xxx CC2xxx CC3xxx RF430xxx

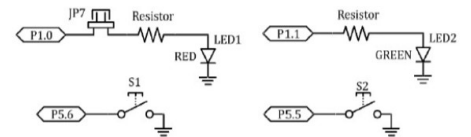
	MSP430FR5994 (MSP43FR59xx)	MSP430F5529 (MSP430F55xx)
Core	TI proprietary 16-bit core	TI proprietary 16-bit core
Active Mode Current	~225 µA at 1 MHz (3.0 V) (*Unified memory, FRAM memory execution)	~360 µA at 1 MHz (3.0 V) (*PMMCOREV0, Flash memory execution)
Low-power Mode 4 (LPM4) Current	~0.6 µA (3.0 V) (*Including SYS, 25 °C)	~1.4 µA (3.0 V) (*PMMCOREV0, 25 °C)



Pull-up or Pull-down Resistor Enable Register

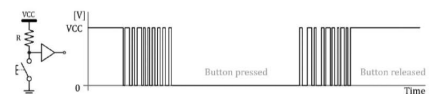
P1REN.y	P1DIR.y	P1OUT.y	I/O configuration
0	0	0	Input
0	0	1	Input
0	1	0	Output
0	1	1	Output
1	0	0	Input / internal pull-down resistor
1	0	1	Input / internal pull-up resistor
1	1	0	Output
1	1	1	Output

LEDs and Buttons on MSP430FR5994 Launchpad

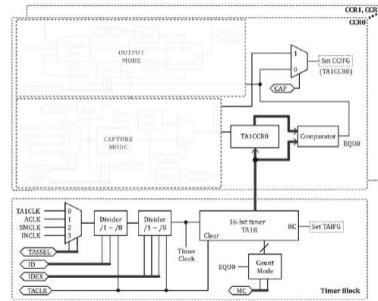


Debouncing Switches

- Non-ideal behavior



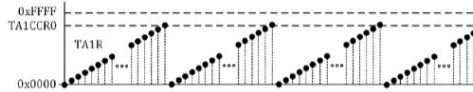
Simplified functional block diagram of Timer_A1



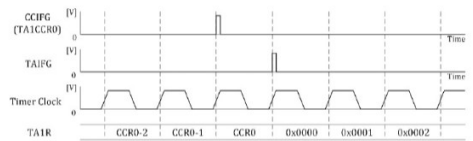
Timer Mode

Mode	MC bits	Description
Stop	0	Timer is halted.
Up	1	Timer counts up to TA1CCR0 value, then, rolls off to zero (Repeatedly)
Continuous	2	Timer counts up to 0xFFFF, then, rolls off to zero (Repeatedly)
Up/Down	3	The timer counts up to the TA1CCR0 value and counts down to zero (Repeatedly)

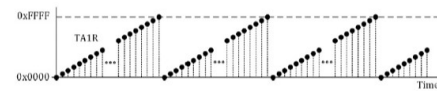
Count pattern for up mode



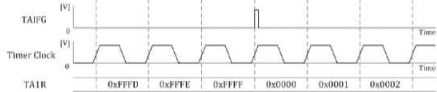
Up mode Flag Setting



Count pattern for Continuous mode



Continuous mode Flag Setting



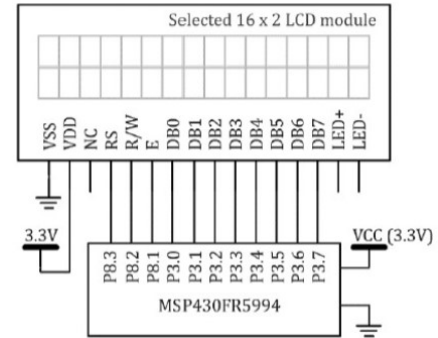
Frequencies of musical notes

Note	Freq. (Hz)	Note	Freq. (Hz)	Note	Freq. (Hz)
C4	261.626	C5	523.251	C6	1046.502
C#4	277.183	C#5	554.365	C#6	1108.731
D4	293.665	D5	587.330	D6	1174.659
D#4	311.127	D#5	622.254	D#6	1244.508
E4	329.628	E5	659.255	E6	1318.510
F4	349.228	F5	698.457	F6	1396.913
F#4	369.994	F#5	739.989	F#6	1479.978
G4	391.995	G5	783.991	G6	1567.982
G#4	415.305	G#5	830.609	G#6	1661.219
A4	440.000	A5	880.000	A6	1760.000
A#4	466.164	A#5	932.328	A#6	1864.656
B4	493.883	B5	987.767	B6	1975.533

Interrupt Sources and Interrupt Vector Addresses

Interrupt Source	Address	Interrupt Source	Address
System Reset	0xFFFE	P2	0xFFD8
System NMI	0xFFFC	TA3	0xFFD6
User NMI	0xFFFA	TA3	0xFFD4
Comparator_E	0xFFFB	P3	0xFFD2
TB0	0xFFFB	P4	0xFFD0
TB0	0xFFFA	RTC_C	0xFFCE
Watchdog timer	0xFFE2	AES	0xFFCC
eUSCI_A0	0xFFD0	TA4	0xFFCB
eUSCI_B0	0xFFEE	TA4	0xFFCA
ADC12_B	0xFFEC	P5	0xFFC6
TA0	0xFFEA	P6	0xFFC4
TA0	0xFFEB	eUSCI_A2	0xFFC2
eUSCI_A1	0xFFE6	eUSCI_A3	0xFFC0
DMA	0xFFE4	eUSCI_B1	0xFFBE
TA1	0xFFE2	eUSCI_B2	0xFFBC
TA1	0xFFE0	eUSCI_B3	0xFFBA
P1	0xFFDE		
TA2	0xFFDC		
TA2	0xFFDA		

LCD Connection & Pin Diagram



Symbol	Pin	Description
VSS	1	Ground
VDD	2	Supply Voltage (+3.3V)
NC	3	No Connect
RS	4	Register Select signal. RS=0: Command, RS=1: Data
R/W	5	Read/Write select signal. R/W=1: Read, R/W=0: Write
E	6	Enable signal (Falling edge triggered)
DB0 ~ DB7	7~14	Data bus lines.
LED+	15	Backlight LED Anode (+3.0V)
LED-	16	Backlight LED Cathode (Ground)

Extra CMDS:

RLA <Register> - Left Shift Register, equivalent to multiplying by 2.

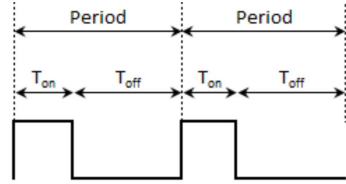
SETZ - Sets a byte register to 1 if Zero Flag is Set, otherwise it sets the register to 0

CMP - Compare

If A=B, Zero Flag (ZF) = 1

If A ≠ B, Zero Flag (ZF)=0

Frequency Formulas:



$$\text{Period} = 1 / \text{Frequency}$$

$$\text{Period} = T_{on} + T_{off}$$

$$\text{Duty Cycle} = T_{on} / (T_{on} + T_{off}) * 100$$

(On Percentage)

$$f_{out} = \frac{f_{timer}}{2 * CCR0}$$

$$f = \frac{1}{\text{period}}$$

Steps for Finding Which Key in the Matrix is Being Pressed:

P4DIR determines if pin is INPUT (0) / OUTPUT (1)
LSB TOP

P4OUT determines if OUTPUT pin is ACTIVE LOW (0) / ACTIVE HIGH (1)

LSB TOP (Useless TBH)

P4IN determines which pin is NOT ACTIVE (0) / ACTIVE (1)
LSB BOT

1. Create a Pin, P4IN, P4DIR chart
2. Convert P4IN and P4DIR to binary
3. Find out which P4DIR INPUT PINS are set to 0 for P4IN
4. Trace path by eliminating which OUTs are 1, whichever is 0,0 is active!

Steps for Writing on LCD:

Step 1: Check LCD_command using the table. IMPORTANT: IGNORE 2 LEFTMOST COLUMNS. LCD_command(0x01) is typically to clear display

Step 2: LCD_command(0xC#) or (0x8#) determines what LCD position the characters will be at

Step 3: First convert 0hC# or 0h8# into 0b11#####, then subtract the MSB1.

Step 4: Convert the subtracted bits into hex and that will be the location of the letters

Steps for Flowchart:

Step 1: Square bracket individual processes on the code block.

Step 2: Use diamond bracket for individual jumps commands.

Step 3: Use these for reference lol:

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

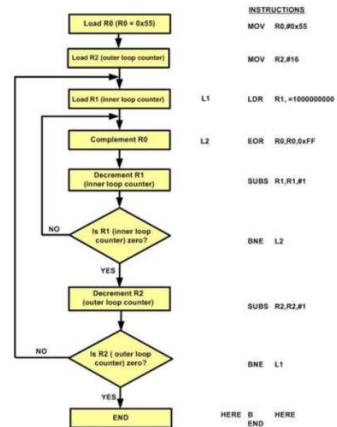


Figure 4- 1: Flowchart for Example 4-3

Program 2.1 - LED Blink Test Program

This program is a basic LED blink example for the MSP430FR5994. It initializes the GPIO, configures the output for P1.0, and toggles the LED state with a delay.

```
#include <msp430.h>

int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // unlock I/O

    P1DIR |= BIT0; // Set P1.0 as output
    P1OUT |= BIT0; // Turn LED on

    while (1) {
        P1OUT ^= BIT0; // Toggle LED
        __delay_cycles(100000);
    }
}
```

Program 4.2 - Assembly LED Blink Program

This program toggles the LED using Assembly.

```
.text
.global _start

_start:
    B15.0 MOVW1, &P1DIR ; Set P1.0 as output
    B15.0 MOVW1, &P1OUT ; Turn on LED

_loop:
    XOR.B #0x01, &P1OUT ; Toggle LED
    MOV.W #20000, R10

_delay:
    DEC.W R10
    JNC _delay
    JMP _loop
```

Program 8.1 - Interrupt-Based Button and LED Program

This program enables an interrupt on a button press to toggle an LED.

```
#include <msp430.h>

void PortISR(void) __attribute__((interrupt(PORTS_VECTOR)));
void PortISR(void) {
    if (P2IFG & BIT6) {
        P2OUT ^= BIT6; // Toggle LED
        __delay_cycles(50000); // debounce delay
        P2IFG &= ~BIT6; // clear interrupt flag
    }
}

int main(void) {
    WDTCTL = WDTPW | WDTHOLD;
    PM5CTL0 &= ~LOCKLPM5;

    P1DIR |= BIT0; // LED output
    P1OUT &= ~BIT0; // button input
    P2REN |= BIT6; // enable pull-up
    P2IE |= BIT6; // enable interrupt
    P2IES |= BIT6; // high-to-low transition
    P2IFG &= ~BIT6; // clear flag

    __enable_interrupt();

    while (1) {
        __no_operation();
    }
}
```

Pulse Generation program, CCIFG based

#include <msp430.h> - SMCLK configured in up mode

```
int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // hold the watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // clear LOCKLPM5 bit
    P1DIR |= BIT0; // output direction (P1.0)
    P1OUT = 0; // output direction (P1.0)
    P2REN |= BIT6; // enable internal resistor
    P2OUT |= BIT6; // pull-up resistor
    P2IES |= BIT6; // interrupt on high-to-low transition
    P2IFG &= ~BIT6; // clear interrupt flag
    P2IE |= BIT6; // enable interrupt (P5.6)
    __enable_interrupt(); // enable general interrupt

    while (1) {
        __no_operation(); // no operation
    }
    return 0;
}
```

Pulse Generation program
#include <msp430.h> TAIFG based

```
int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // hold the watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // clear LOCKLPM5 bit
    P1DIR |= BIT0; // output direction (P1.0)
    TA1CCR0 = 2000; // TA1CCR0 value
    TA1CTL = TASSEL_2 | MC_1 | TACLR; // TA1CTL setup

    while (1) {
        if ((TA1CTL & TAIFG) != 0) { // check whether TAIFG is set
            P1OUT ^= BIT0; // toggle (P1.0)
            P1OUT = 0; // clear (P1.0)
            TA1CTL &= ~TAIFG; // clear TAIFG flag
        }
    }
}
```

This program initializes an LCD and displays text.

```
#include <msp430.h>

void LCD_command(unsigned char in);
void LCD_write(unsigned char in);
void LCD_init(void);

int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // hold watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // clear LOCKLPM5 bit

    P3DIR |= BIT0; // Set P3 as output (for LCD data)
    P3OUT = 0x00; // Clear P3 output

    P3DIR |= BIT5; // Set PB.1, PB.2, PB.3 as output (LCD control pins)
    P3OUT = 0x00; // Clear control signals

    LCD_init(); // Initialize LCD

    // Display "Text" on LCD
    LCD_command("T");
    LCD_write("T");
    LCD_write("a");
    LCD_write("s");
    LCD_write("t");

    while (1) {
        __no_operation();
    }
    return 0;
}

void LCD_command(unsigned char in) {
    P3OUT = 0;
    P3OUT |= BIT3; // Clear RS (Command mode)
    P3OUT &= ~BIT2; // Clear R/W (Write mode)
    P3OUT |= BIT1; // Set E (Enable)
    __delay_cycles(100);
    P3OUT &= ~BIT1; // Clear E (Disable)
}

void LCD_write(unsigned char in) {
    P3OUT = 0;
    P3OUT |= BIT3; // Set RS (Data mode)
    P3OUT &= ~BIT2; // Clear R/W (Write mode)
    P3OUT |= BIT1; // Set E (Enable)
    __delay_cycles(100);
    P3OUT &= ~BIT1; // Clear E (Disable)
}

void LCD_init(void) {
    P3OUT &= ~BIT1; // Clear E (Ensure disabled initially)
    __delay_cycles(15000); // Initial delay

    LCD_command(0x30); // Wake up
    __delay_cycles(100);
    LCD_command(0x30); // Repeat wake up
    __delay_cycles(100);
    LCD_command(0x30); // Repeat wake up
    __delay_cycles(100);

    LCD_command(0x38); // Function set: 8-BIT mode, 2-line display
    LCD_command(0x10); // Set cursor
    LCD_command(0x0C); // Display ON, cursor OFF
    LCD_command(0x06); // Entry mode set
    LCD_command(0x01); // Clear display
    __delay_cycles(1000);
}
```

Interrupt LED program

```
#include <msp430.h>
int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // hold the watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // clear LOCKLPM5 bit
    P1DIR |= BIT0; // output direction (P1.0)
    P1OUT = 0; // output direction (P1.0)
    P2REN |= BIT6; // enable internal resistor
    P2OUT |= BIT6; // pull-up resistor
    P2IES |= BIT6; // interrupt on high-to-low transition
    P2IFG &= ~BIT6; // clear interrupt flag
    P2IE |= BIT6; // enable interrupt (P5.6)
    __enable_interrupt(); // enable general interrupt

    while (1) {
        __no_operation(); // no operation
    }
    return 0;
}

#pragma vector=PORTS_VECTOR
__interrupt void PortISR_handler(void) {
    if (P2IFG & BIT6) {
        P1OUT ^= BIT0; // toggle (P1.0)
        __delay_cycles(500); // debounce
        P2IFG &= ~BIT6; // clear interrupt flag
    }
}
```

Interrupt PWM program
- CCIFG

```
#include <msp430.h>
int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // hold the watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // clear LOCKLPM5 bit
    P1DIR |= BIT0; // output direction (P1.0)
    P1OUT = 0; // output direction (P1.0)
    P2REN |= BIT6; // enable internal resistor
    P2OUT |= BIT6; // pull-up resistor
    P2IES |= BIT6; // interrupt on high-to-low transition
    P2IFG &= ~BIT6; // clear interrupt flag
    P2IE |= BIT6; // enable interrupt (P5.6)
    __enable_interrupt(); // enable general interrupt

    while (1) {
        P1OUT ^= BIT0; // toggle (P1.0)
        __delay_cycles(250000); // delay
    }
    return 0;
}

#pragma vector = TIMER1_A0_VECTOR
__interrupt void Timer1_A0_ISR(void) {
    P1OUT ^= BIT0; // toggle (P1.0)
}
```